Playwright API Mocking: Scenarios with TypeScript Code

Assumptions: `baseURL` is set in `playwright.config.ts`. URLs, labels and endpoints are examples; replace them with your app's.

Golden rule: register `page.route()` BEFORE the action that triggers the request (before `goto` or the click).

---

1. Happy path: return fixed JSON

ts

```ts
import { test, expect } from '@playwright/test';

test('list renders mocked products', async ({ page }) => {
  await page.route('**/api/products', route =>
    route.fulfill({
      status: 200,
      json: [
        { id: 1, name: 'Laptop', price: 900 },
        { id: 2, name: 'Mouse', price: 20 },
      ],
    })
  );

  await page.goto('/products');
  await expect(page.getByRole('row')).toHaveCount(3); // header + 2 rows
  await expect(page.getByText('Laptop')).toBeVisible();
});
```

2. Empty state

ts

```ts
test('shows empty state when no products', async ({ page }) => {
  await page.route('**/api/products', route => route.fulfill({ json: [] }));
  await page.goto('/products');
  await expect(page.getByText('No products found')).toBeVisible();
});
```

3. Server error (500)

ts

```ts
test('shows error banner on 500', async ({ page }) => {
  await page.route('**/api/products', route =>
    route.fulfill({ status: 500, json: { message: 'Internal Server Error' } })
  );
  await page.goto('/products');
  await expect(page.getByRole('alert')).toContainText('Something went wrong');
});
```

4. Client errors: 400, 401, 403, 404, 429

ts

```ts
const errorCases = [
  { status: 400, msg: 'Bad request' },
  { status: 401, msg: 'Please log in again' },
  { status: 403, msg: 'You do not have access' },
  { status: 404, msg: 'Not found' },
  { status: 429, msg: 'Too many requests' },
];

for (const { status, msg } of errorCases) {
  test(`handles HTTP ${status}`, async ({ page }) => {
    await page.route('**/api/profile', route =>
      route.fulfill({ status, json: { message: msg } })
    );
    await page.goto('/profile');
    await expect(page.getByRole('alert')).toContainText(msg);
  });
}
```

5. Network failure (offline / DNS / connection refused)

ts

```ts
test('handles network failure', async ({ page }) => {
  await page.route('**/api/products', route => route.abort('failed'));
  await page.goto('/products');
  await expect(page.getByText('Unable to connect')).toBeVisible();
});
```

Other abort reasons: `'timedout'`, `'connectionrefused'`, `'internetdisconnected'`, `'aborted'`.

6. Slow response / loading spinner

```ts
test('shows spinner while loading', async ({ page }) => {
  await page.route('**/api/products', async route => {
    await new Promise(r => setTimeout(r, 3000)); // simulate slow API
    await route.fulfill({ json: [{ id: 1, name: 'Laptop' }] });
  });

  await page.goto('/products');
  await expect(page.getByTestId('spinner')).toBeVisible();
  await expect(page.getByText('Laptop')).toBeVisible();
  await expect(page.getByTestId('spinner')).toBeHidden();
});
```

7. Timeout handling (request never completes)

```ts
test('shows timeout message', async ({ page }) => {
  await page.route('**/api/report', route => route.abort('timedout'));
  await page.goto('/reports');
  await expect(page.getByText('Request timed out')).toBeVisible();
});
```

8. Modify a real response (partial mock)

```ts
test('override one field from real response', async ({ page }) => {
  await page.route('**/api/products', async route => {
    const response = await route.fetch();
    const data = await response.json();
    data[0].price = 0;                    // tweak real data
    await route.fulfill({ response, json: data });
  });

  await page.goto('/products');
  await expect(page.getByRole('row').nth(1)).toContainText('0');
});
```

9. Modify the outgoing request (headers or body)

```ts
```

```ts
test('add header to outgoing request', async ({ page }) => {
  await page.route('**/api/orders', route =>
    route.continue({ headers: { ...route.request().headers(), 'x-env': 'test' }
  );
  await page.goto('/orders');
});

test('change request payload', async ({ page }) => {
  await page.route('**/api/orders', route => {
    const body = route.request().postDataJSON();
    route.continue({ postData: JSON.stringify({ ...body, qty: 99 }) });
  });
});
```

10. Different behaviour per HTTP method

ts

```ts
test('mock GET and POST on same endpoint', async ({ page }) => {
  await page.route('**/api/users', async route => {
    const req = route.request();
    if (req.method() === 'GET') {
      return route.fulfill({ json: [{ id: 1, name: 'Asha' }] });
    }
    if (req.method() === 'POST') {
      return route.fulfill({ status: 201, json: { id: 99, ...req.postDataJSON()
    }
    return route.continue();
  });

  await page.goto('/users');
  await expect(page.getByText('Asha')).toBeVisible();
});
```

11. Verify the request the UI sends (request assertion)

ts

```ts
test('form posts correct payload', async ({ page }) => {
  await page.route('**/api/orders', route =>
    route.fulfill({ status: 201, json: { id: 1 } })
  );
  await page.goto('/orders/new');
  await page.getByLabel('Item').fill('Book');
  await page.getByLabel('Quantity').fill('2');

  const [request] = await Promise.all([
    page.waitForRequest('**/api/orders'),
    page.getByRole('button', { name: 'Place order' }).click(),
  ]);

  expect(request.method()).toBe('POST');
  expect(request.postDataJSON()).toEqual({ item: 'Book', qty: 2 });
  await expect(page.getByText('Order placed')).toBeVisible();
});
```

12. Query params: match and respond by params (pagination, search)

ts

```ts
test('pagination uses page param', async ({ page }) => {
  await page.route('**/api/products?*', route => {
    const url = new URL(route.request().url());
    const pageNo = Number(url.searchParams.get('page') ?? 1);
    const items = pageNo === 1
      ? [{ id: 1, name: 'Item 1' }, { id: 2, name: 'Item 2' }]
      : [{ id: 3, name: 'Item 3' }];
    route.fulfill({ json: { items, totalPages: 2 } });
  });

  await page.goto('/products');
  await expect(page.getByText('Item 1')).toBeVisible();
  await page.getByRole('button', { name: 'Next' }).click();
  await expect(page.getByText('Item 3')).toBeVisible();
});
```

13. Dynamic URL with ID (regex)

ts

```ts
test('mock user by id', async ({ page }) => {
  await page.route(/\/api\/users\/\d+$/, route => {
    const id = route.request().url().split('/').pop();
    route.fulfill({ json: { id: Number(id), name: `User ${id}` } });
  });

  await page.goto('/users/42');
  await expect(page.getByText('User 42')).toBeVisible();
});
```

14. Different response on each call (retry, polling, status change)

ts

```ts
test('first call fails, retry succeeds', async ({ page }) => {
  let calls = 0;
  await page.route('**/api/products', route => {
    calls++;
    if (calls === 1) return route.fulfill({ status: 500 });
    return route.fulfill({ json: [{ id: 1, name: 'Laptop' }] });
  });

  await page.goto('/products');
  await page.getByRole('button', { name: 'Retry' }).click();
  await expect(page.getByText('Laptop')).toBeVisible();
  expect(calls).toBe(2);
});

test('job status changes over polling', async ({ page }) => {
  const statuses = ['PENDING', 'RUNNING', 'DONE'];
  let i = 0;
  await page.route('**/api/jobs/1', route =>
    route.fulfill({ json: { status: statuses[Math.min(i++, statuses.length - 1)
  );
  await page.goto('/jobs/1');
  await expect(page.getByTestId('status')).toHaveText('DONE', { timeout: 15_000
});
```

15. Use `route.fallback()` for layered handlers

Handlers run in reverse registration order. Use `fallback()` to pass control to an earlier handler.

ts

```ts
test('global mock + per-test override', async ({ page }) => {
  await page.route('**/api/**', route => route.fulfill({ json: { ok: true } }))
  await page.route('**/api/products', route => {
    if (route.request().method() === 'GET') {
      return route.fulfill({ json: [{ id: 1 }] });
    }
    return route.fallback(); // let the default handler deal with it
  });
});
```

16. Serve responses from a JSON fixture file

ts

```ts
import users from '../test-data/users.json';

test('users from fixture file', async ({ page }) => {
  await page.route('**/api/users', route => route.fulfill({ json: users }));
  await page.goto('/users');
  await expect(page.getByRole('row')).toHaveCount(users.length + 1);
});

// Or serve the file directly:
// route.fulfill({ path: 'test-data/users.json', contentType: 'application/json
```

17. Record and replay with a HAR file

ts

```ts
// Run once to record (set update: true)
test('record HAR', async ({ page }) => {
  await page.routeFromHAR('hars/products.har', { url: '**/api/**', update: true
  await page.goto('/products');
});

// Replay in normal runs
test('replay HAR', async ({ page }) => {
  await page.routeFromHAR('hars/products.har', {
    url: '**/api/**',
    update: false,
    notFound: 'fallback', // unmatched requests go to the real network
  });
  await page.goto('/products');
  await expect(page.getByText('Laptop')).toBeVisible();
});
```

18. GraphQL: mock by operation name

ts

```ts
test('mock GraphQL query', async ({ page }) => {
  await page.route('**/graphql', route => {
    const { operationName } = route.request().postDataJSON();
    if (operationName === 'GetUsers') {
      return route.fulfill({
        json: { data: { users: [{ id: '1', name: 'Asha' }] } },
      });
    }
    if (operationName === 'CreateUser') {
      return route.fulfill({
        json: { data: { createUser: { id: '2', name: 'Ravi' } } },
      });
    }
    return route.continue();
  });

  await page.goto('/users');
  await expect(page.getByText('Asha')).toBeVisible();
});
```

19. Authentication: mock login and token expiry

ts

```ts
test('mock login success', async ({ page }) => {
  await page.route('**/api/login', route =>
    route.fulfill({ json: { token: 'fake-jwt-token', user: { name: 'Asha' } } })
  );
  await page.goto('/login');
  await page.getByLabel('Email').fill('a@b.com');
  await page.getByLabel('Password').fill('any');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page).toHaveURL(/dashboard/);
});

test('expired token redirects to login', async ({ page }) => {
  await page.route('**/api/me', route =>
    route.fulfill({ status: 401, json: { message: 'Token expired' } })
  );
  await page.goto('/dashboard');
  await expect(page).toHaveURL(/login/);
});
```

20. Third-party calls: block analytics, ads, chat widgets

ts

```ts
test('block third-party scripts', async ({ page }) => {
  await page.route(/google-analytics|doubleclick|hotjar|intercom/, route => rou
  await page.goto('/');
});

// Block heavy resources to speed up tests
test('block images and fonts', async ({ page }) => {
  await page.route('**/*', route => {
    const type = route.request().resourceType();
    return ['image', 'font', 'media'].includes(type) ? route.abort() : route.co
  });
});
```

21. Mock for the whole context (popups, multiple tabs)

ts

```ts
test('mock applies to popup too', async ({ context, page }) => {
  await context.route('**/api/config', route => route.fulfill({ json: { theme:
  await page.goto('/');
  const [popup] = await Promise.all([
    context.waitForEvent('page'),
    page.getByRole('link', { name: 'Open report' }).click(),
  ]);
  await expect(popup.locator('body')).toHaveClass(/dark/);
});
```

22. File download / upload endpoint

ts

```ts
test('mock file download', async ({ page }) => {
  await page.route('**/api/export', route =>
    route.fulfill({
      status: 200,
      headers: { 'content-disposition': 'attachment; filename="report.csv"' },
      contentType: 'text/csv',
      body: 'Name,Email\nAsha,a@b.com',
    })
  );

  await page.goto('/reports');
  const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.getByRole('button', { name: 'Export CSV' }).click(),
  ]);
  expect(download.suggestedFilename()).toBe('report.csv');
});

test('mock upload failure', async ({ page }) => {
  await page.route('**/api/upload', route =>
    route.fulfill({ status: 413, json: { message: 'File too large' } })
  );
  await page.goto('/upload');
  await page.setInputFiles('input[type=file]', 'test-data/sample.pdf');
  await expect(page.getByText('File too large')).toBeVisible();
});
```

23. Response headers and cookies

ts

```ts
test('mock response headers', async ({ page }) => {
  await page.route('**/api/products', route =>
    route.fulfill({
      status: 200,
      headers: {
        'x-total-count': '42',
        'set-cookie': 'session=abc123; Path=/',
      },
      json: [],
    })
  );
  await page.goto('/products');
  await expect(page.getByText('42 results')).toBeVisible();
});
```

24. WebSocket mocking (Playwright 1.48+)

ts

```ts
test('mock websocket', async ({ page }) => {
  await page.routeWebSocket('wss://example.com/ws', ws => {
    ws.onMessage(message => {
      if (message === 'ping') ws.send('pong');
    });
  });
  await page.goto('/live');
  await expect(page.getByText('Connected')).toBeVisible();
});
```

25. Assert a request was NOT made

ts

```ts
test('no API call when form invalid', async ({ page }) => {
  let called = false;
  await page.route('**/api/orders', route => {
    called = true;
    route.fulfill({ status: 201, json: {} });
  });

  await page.goto('/orders/new');
  await page.getByRole('button', { name: 'Place order' }).click();
  await expect(page.getByText('Item is required')).toBeVisible();
  expect(called).toBe(false);
});
```

26. Count calls and verify caching / debounce

ts

```ts
test('search is debounced', async ({ page }) => {
  let count = 0;
  await page.route('**/api/search*', route => {
    count++;
    route.fulfill({ json: [] });
  });

  await page.goto('/search');
  await page.getByPlaceholder('Search').pressSequentially('playwright', { delay
  await page.waitForResponse('**/api/search*');
  expect(count).toBeLessThanOrEqual(2);
});
```

27. Remove a mock mid-test

ts

```ts
test('switch from mocked to real', async ({ page }) => {
  const handler = (route: any) => route.fulfill({ json: [] });
  await page.route('**/api/products', handler);
  await page.goto('/products');
  await expect(page.getByText('No products found')).toBeVisible();

  await page.unroute('**/api/products', handler);
  await page.reload(); // now hits the real API
});
```

28. Reusable mock helper

ts

```ts
// utils/mockApi.ts
import { Page } from '@playwright/test';

export async function mockJson(page: Page, url: string, data: unknown, status =
  await page.route(url, route => route.fulfill({ status, json: data }));
}

export async function mockError(page: Page, url: string, status = 500, message
  await page.route(url, route => route.fulfill({ status, json: { message } }));
}

// usage
// await mockJson(page, '**/api/products', [{ id: 1 }]);
// await mockError(page, '**/api/products', 503, 'Service unavailable');
```

29. Auto-applied mocks through a fixture

ts

```ts
// fixtures/index.ts
import { test as base } from '@playwright/test';

export const test = base.extend({
  page: async ({ page }, use) => {
    await page.route(/google-analytics|hotjar/, route => route.abort());
    await page.route('**/api/feature-flags', route =>
      route.fulfill({ json: { newCheckout: true } })
    );
    await use(page);
  },
});
export { expect } from '@playwright/test';
```

30. Mock and API test in one project (what is NOT intercepted)

`page.route` only intercepts browser traffic. The `request` fixture is never mocked, so use it to hit the real API.

ts

```
test('real API check', async ({ request }) => {
  const res = await request.get('/api/health');
  expect(res.ok()).toBeTruthy();
});
```

Common mistakes

Mistake	Fix
Route registered after `goto()` or the click	Register before the triggering action
Pattern doesn't match (query string, domain)	Use `**/api/products**` or a regex; log `request.url()` to debug
Forgetting `await route.fulfill()`	Always await or return it
Handler never calls fulfill/continue/abort	The request hangs; always resolve the route
Mocking everything	Keep some real end-to-end tests
Expecting `page.route` to affect `request` fixture	It does not; they are separate

Quick reference

Goal	Method
Return fake data	`route.fulfill({ json })`
Fail the request	`route.abort('failed')`
Let it through (optionally edited)	`route.continue({ headers, postData })`
Call real server, then edit	`route.fetch()` then `route.fulfill({ response, json })`
Defer to earlier handler	`route.fallback()`
Record/replay	`page.routeFromHAR()`
WebSocket	`page.routeWebSocket()`
Remove mock	`page.unroute()`